

alfe AutoLISP CLI front-end

USER MANUAL

clautolisp contributors

June 24, 2026

Contents

1	Introduction	1
2	Installation	1
3	Backend selection	2
3.1	Algorithm	2
3.2	Why not auto-detect?	2
3.3	Pinning a default backend per shell	3
4	Dialect selection	3
5	Host selection	3
6	Actions	4
6.1	Action output semantics	4
6.2	Interactive REPL on CAD backends	5
7	Encoding	5
8	Init files	6
9	The AUTOLISP- globals	6
10	Troubleshooting	6
10.1	alfe prints "Backend ... is not registered"	6
10.2	alfe --bricscad fails on Linux	7
10.3	alfe --autocad fails on macOS or Linux	7
10.4	"alfe with no flags is suddenly different"	7

11 See also	7
12 Synchronisation	7

1 Introduction

`alfe(1)` is the unified command-line front-end for AutoLISP. It drives three backends behind a single command-line shape:

- `clautolisp` an in-process AutoLISP evaluator that ships with `alfe`. No CAD required. The default backend.
- `bricscad` BricsCAD V26+, via batch (`-b run.scr`) or COM /AppleScript automation.
- `autocad` AutoCAD 2026+, via `accorreconsole` batch or COM automation (MS-Windows only).

The same `-x` `EXPR`, `-l` `FILE`, `--dialect` `NAME` and `--host` `MODE` options work against every backend. Programs written for `alfe` run unchanged whether you point them at the local `clautolisp` engine or at a real CAD session.

2 Installation

The build target produces two binaries: `alfe-sbcl` (preferred, faster startup) and `alfe-ccl` (Clozure CL backup, when SBCL is unavailable). The `install` rule places them under `$(PREFIX)/bin/` along with a wrapper `alfe` that resolves to one or the other:

```
cd autolisp-front-end
make build
sudo make install PREFIX=/opt/local
```

After install:

<code>alfe --help</code>	# full option reference
<code>alfe -x "(print (+ 1 2))"</code>	# one-shot eval, prints "3" then exits
<code>alfe -x "(+ 1 2)"</code>	# silent see "Action output semantics" below
<code>alfe -l script.lsp</code>	# load a file, default backend
<code>alfe</code>	# interactive REPL, default backend

The `--clautolisp` / `--bricscad` / `--autocad` flags select a specific backend; absent a flag, the in-process clautolisp engine is used (see Backend selection).

3 Backend selection

When no backend flag is given, `alfe` defaults to the `clautolisp` in-process engine. This is a change from the historical auto-detection: CAD backends are now NEVER selected automatically they require an explicit `--autocad` / `--bricscad` flag (or an environment override).

3.1 Algorithm

```
if --bricscad, --autocad or --clautolisp is explicit:
    use that backend
else if $ALFE_BACKEND_OVERRIDE is set:
    use that backend
else if $AUTOLISP_BACKEND is set:
    use that backend (legacy alias)
else:
    backend = clautolisp      ; default on every system
```

3.2 Why not auto-detect?

A host with both BricsCAD and AutoCAD installed got order-dependent results from the historical auto-detect (whichever backend's `DETECT` method was probed first won). With the new rule, `alfe` with no flags always means "talk to the in-process clautolisp engine" no probing, no surprises, no dependency on CAD installation state.

3.3 Pinning a default backend per shell

Users who want a CAD backend selected by default can set `$ALFE_BACKEND_OVERRIDE` once in their shell init:

```
# ~/.zshrc or ~/.bashrc
export ALFE_BACKEND_OVERRIDE=bricscad
```

Explicit flags still win `alfe --clautolisp -x "(+ 1 2)"` runs under clautolisp even with the override set.

4 Dialect selection

The `--dialect NAME` option (and its `--strict` / `--autocad` / `--bricscad` / `--clautolisp` shorthands) selects the AutoLISP dialect descriptor the runtime uses: which reader knobs are on, which encoding default applies, which special-case syntax is accepted.

Option	Dialect	Notes
<code>--dialect strict</code> / <code>--strict</code>	<code>strict</code>	Default. Portable AutoCAD BricsCAD
<code>--dialect autocad-2026</code> / <code>--autocad</code>	<code>autocad-2026</code>	AutoCAD 2026 surface. UTF-8 default
<code>--dialect bricscad-v26</code> / <code>--bricscad</code>	<code>bricscad-v26</code>	BricsCAD V26. Lax reader, hex-float at
<code>--dialect clautolisp</code> / <code>--clautolisp</code>	<code>clautolisp</code>	Superset of strict + clautolisp extension

The dialect is independent of the backend you can run `alfe --clautolisp --dialect bricscad-v26 script.lsp` to evaluate under the BricsCAD reader rules in the in-process engine, which is useful for testing.

Whatever the dialect selected, all operators remain callable. Out-of-dialect operators are eligible for a diagnostic warning at use time (the warning machinery itself is tracked separately see `deferred-clautolisp-out-of-dialect-warnings.issu` in the project issue tracker).

5 Host selection

`--host MODE` picks the HAL (host abstraction layer) backend the clautolisp engine talks to. Defaults to `mock` (a recording host that consumes `--mock-input PATH` for `GETSTRING` / `GETPOINT` prompts).

Option	Host
<code>--host mock</code>	MockHost (default). Drives prompts from a file or stdin.
<code>--host null</code>	NullHost. All host calls return nil immediately.

For the CAD backends the host is the CAD itself `--host` has no effect there.

6 Actions

alfe accepts an ordered queue of actions on the command line. The queue runs in order then the engine terminates (or enters the REPL with `-i`).

Option	Action
<code>-x</code> <code>EXPR</code> / <code>--eval</code> <code>EXPR</code>	Evaluate <code>EXPR</code> . Value is NOT auto-printed wrap with <code>(print)</code> if you want
<code>-l</code> <code>FILE</code> / <code>--load</code> <code>FILE</code>	Load and evaluate <code>FILE</code> .
<code>FILE.lsp</code> (positional)	Equivalent to <code>-l FILE.lsp</code> .
<code>-i</code> / <code>--interactive</code>	Enter REPL after the queue (same context). Auto-printing is enabled in this context
<code>--quit</code>	Force engine shutdown after the queue.
<code>--dry-run</code>	Print the resolved plan; don't run anything.

Examples:

```

alfe -x "(print (+ 1 2))"           # 3
alfe -x "(+ 1 2)"                  # silent value discarded
alfe -l prelude.lsp -x "(my-main)"  # load then call
alfe -l script.lsp -i              # load then drop to REPL
alfe --bricscad -l ../scripts/draw.lsp # run under BricsCAD
alfe --dry-run -l foo.lsp          # show the plan only

```

6.1 Action output semantics

`-x` `EXPR`, `-l` `FILE` and `--main` `FN` are **batch** evaluations, not REPL steps. The value an action produces is **not** echoed to `alfe`'s `stdout` use an explicit printing primitive in the expression `((print ))`, `(princ )`, `(prin1 )` when you want the value to come back. This is uniform across the `--clautolisp`, `--bricscad` and `--autocad` backends.

The interactive REPL (`-i` / `--interactive`) is the only context where values are auto-printed: there, each top-level form's value is read, evaluated and printed (read-eval-print loop). `-x` and `-l` actions issued alongside `-i` remain silent-by-default; only forms typed AT the REPL prompt print their values.

If you want a one-line shorthand for "evaluate and print", define a helper in your `init` file:

```

(defun ep (form) (print (eval form)))
;; then
alfe -l ~/.autolisp -x "(ep (quote (list 1 2 3)))"

```

6.2 Interactive REPL on CAD backends

`alfe --interactive --bricscad` (and `--autocad`) drops into a live REPL backed by the same file-IPC protocol the batch `-x` / `-l` actions use. Practical points:

- The prompt is `alfe>` (or `= >=` for continuation lines when the form isn't balanced yet). Multi-line forms are accepted the reader waits for a balanced parenthesis count before forwarding.
- Each form is sent through `protocol/stdin.txt`; output is drained live to your terminal as the CAD prints (no end-of-plan buffering).
- Errors do not exit the REPL: a failed eval surfaces its message on `stderr`, the prompt comes back, the runtime stays in its read-eval loop.
- Exit with any of: Ctrl-D (EOF), an empty line that equals `:quit`, `:q`, `:exit`, `(quit)`, `(exit)`, or by queuing a `--quit` action.

You can mix `-l / -x` with `--interactive` in one invocation: the batch actions run first, then control hands off to the REPL with the same shared evaluation context.

7 Encoding

Option	Effect
<code>-e ENC</code>	Override the source-file encoding for the whole session.
<code>-E ENC</code>	Declared terminal-IO encoding; surfaced to AutoLISP code.

Accepted ENC values: `utf-8`, `iso-8859-1` (= `latin-1`), `windows-1252` (= `cp1252`), `us-ascii`. Suffixes select a line-ending variant: `-mac / -cr` for CR, `-dos / -crlf` for CRLF, `-unix / -lf` for LF. So `-e utf-8-crlf` reads a UTF-8 file with CRLF terminators.

Without `-e`, the dialect's default applies (`iso-8859-1` for `strict`, `utf-8` for `autocad-2026 / bricscad-v26 / clautolisp`). When neither dialect nor option specifies one, the process locale (`$LC_ALL`, then `$LANG`, then `$LC_CTYPE`) supplies the fallback.

`--list-encodings` prints every accepted ENC name and exits.

8 Init files

On startup (unless `--no-init / -norc` is given), `alfe` loads the first init file it finds in this order:

1. `~/.alfe` or `~/.alferc`

2. `~/.autolisp` or `~/.autolisprc`
3. `~/.config/alfe/init`
4. `~/.config/autolisp/init`

The `init` file runs under the selected dialect and backend; it's a natural place to put `alref-set-root` calls, custom helpers, REPL preferences, etc.

9 The AUTOLISP- globals

User AutoLISP code can read these globals to branch on session state:

Variable	Meaning
<code>*AUTOLISP-FRONTEND*</code>	ALFE when <code>alfe</code> is the entry point, CLAUTOLISP when <code>clautolisp</code> is
<code>*AUTOLISP-BACKEND*</code>	The engine running user code: CLAUTOLISP / BRICSCAD / AUTOCAD
<code>*AUTOLISP-HELP*</code>	The <code>-help</code> banner of the front-end as a string.
<code>*AUTOLISP-DIALECT*</code>	<code>:strict</code> / <code>:autocad-2026</code> / <code>:bricscad-v26</code> / <code>:clautolisp</code> .
<code>*AUTOLISP-HOST*</code>	<code>:mock</code> / <code>:null</code> / a CAD-specific keyword.
<code>*AUTOLISP-FILE-ENCODING*</code>	Active source-file encoding (always a string). Empty string <code>""</code> means
<code>*AUTOLISP-TERMINAL-ENCODING*</code>	Active terminal encoding (always a string, never empty) falls back to
<code>*AUTOLISP-VERSION*</code>	Version string of the running engine.
<code>*AUTOLISP-LOAD-PATHNAME*</code>	Set by <code>LOAD</code> during evaluation of the loaded file.
<code>*AUTOLISP-EXPRESSION*</code>	Set during evaluation of a <code>-x</code> expression.
<code>*AUTOLISP-INTERACTIVE*</code>	T when the REPL is active, NIL otherwise.

The specifications document the full set and their lifetimes.

10 Troubleshooting

10.1 `alfe` prints "Backend ... is not registered"

You selected an explicit backend that isn't compiled into this build. Usually means a CAD backend is referenced on a host build that doesn't ship it. Pick `--clautolisp` or omit the flag for the default.

10.2 `alfe --bricscad` fails on Linux

BricsCAD ships for Linux but the wrapper integration is currently macOS- and Windows-only. Track `alfe-bricscad-linux.issue`.

10.3 `alfe --autocad` fails on macOS or Linux

AutoCAD is not distributed for those platforms. `alfe` exits with code 3 and a clear message. Use `--bricscad` (where supported) or `--clautolisp` instead.

10.4 "alfe with no flags is suddenly different"

Yes see the Backend selection section. Auto-detection of installed CADs has been removed. Set `$ALFE_BACKEND_OVERRIDE` in your shell init if you want a CAD backend by default.

11 See also

- `alfe(1)` short man page summarising the same options.
- `alfe--specifications.org` the full specification (algorithms, exit codes, file-protocol wire format, per-backend constraints).
- `clautolisp(1)` / `clautolisp-user-manual.org` the in-process engine that powers the `--clautolisp` backend.

12 Synchronisation

Per the project-wide synchronisation rule (`AGENTS.md`, Documentation Synchronisation section), changes to the code, the alfe specifications, this user-manual, or the man page must be propagated to every track.